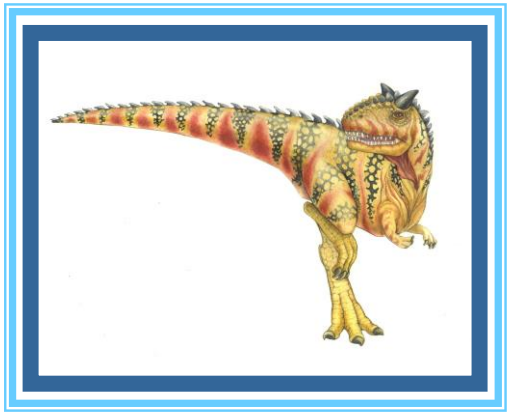
Chapter 3: Processes

فصل سوم: فرایندها



Silberschatz, Galvin and Gagne ©2013

Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Examples of IPC Systems
- Communication in Client-Server Systems



Objectives

اهداف فصل : در پایان این فصل باید بتوانید:

1. مفهوم Process را تعریف کنید.
2. وضعیت‌های فرآیند (Process States) را توضیح دهید.
3. ساختار PCB (Process Control Block) را بشناسید.
4. نحوهٔ ایجاد و خاتمه‌دهی فرآیندها را درک کنید.
5. روش‌های ارتباط بین فرآیندها (IPC) را مقایسه کنید.
6. مدل کلاینت/سرور و روش‌های پیام‌رسانی را توضیح دهید.

- فرآیند مهم‌ترین واحد کاری در سیستم‌عامل است.



Process Concept

- فرآیند =برنامه در حال اجرا
- برنامه (Program) یک موجودیت ثابت و غیرفعال است که روی دیسک ذخیره شده است.
  - مثال: فایل "chrome.exe"
- فرآیند یک موجودیت پویا و فعال است؛ یعنی همان برنامه زمانی که در حافظه (RAM)بارگذاری شده و توسط CPUاجرا می‌شود.
- یک برنامه زمانی به فرآیند تبدیل می‌شود که:

1. به حافظه بارگذاری شود
2. شمارندهٔ برنامه (Program Counter) مقداردهی شود
3. یک ساختار PCB (Process Control Block)برای آن ساخته شود
4. CPUبه آن زمان اجرا بدهد.



Process Concept

- اجرای دستورات یک فرآیند به‌صورت ترتیبی (Sequential) پیش می‌رود.
- ✗ در یک فرآیند واحد اجرای موازی دستورات وجود ندارد.
- ✓ چند فرآیند مختلف می‌توانند هم‌زمان اجرا شوند (Multiprocessing).

- یک برنامه می‌تواند همزمان چندین بار اجرا شود و در نتیجه چند فرایند مختلف ایجاد کند.
  - مثال: اجرای چند پنجره‌ی Chrome مختلف.

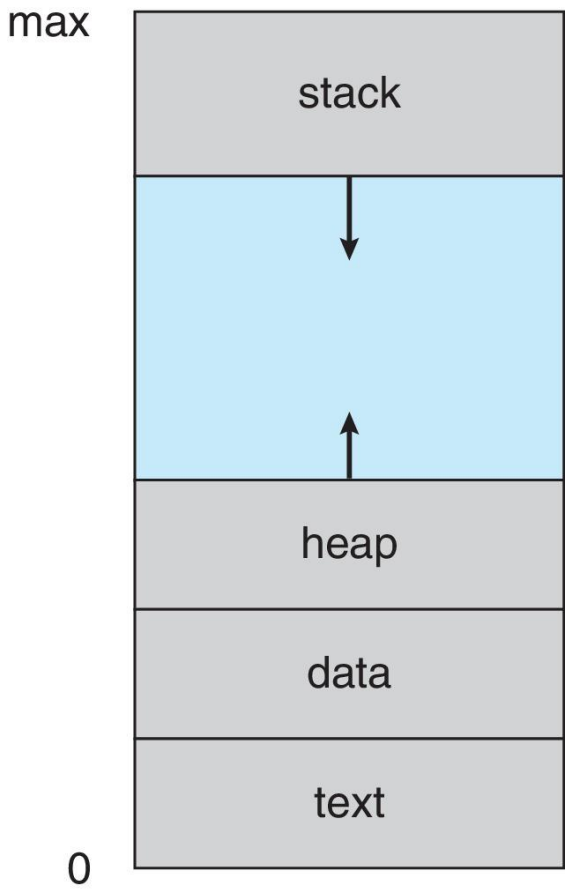


Components of a Process

- یک فرآیند شامل چند بخش است:
  - Text Section: کد برنامه
  - Current Activity: شمارنده برنامه + ثبات‌ها
  - Stack: داده‌های موقت شامل
    - پارامتر توابع
    - آدرس بازگشت
    - متغیرهای محلی
  - Data Section: متغیرهای سراسری
  - Heap: حافظه پویا حین اجرا



Process in Memory

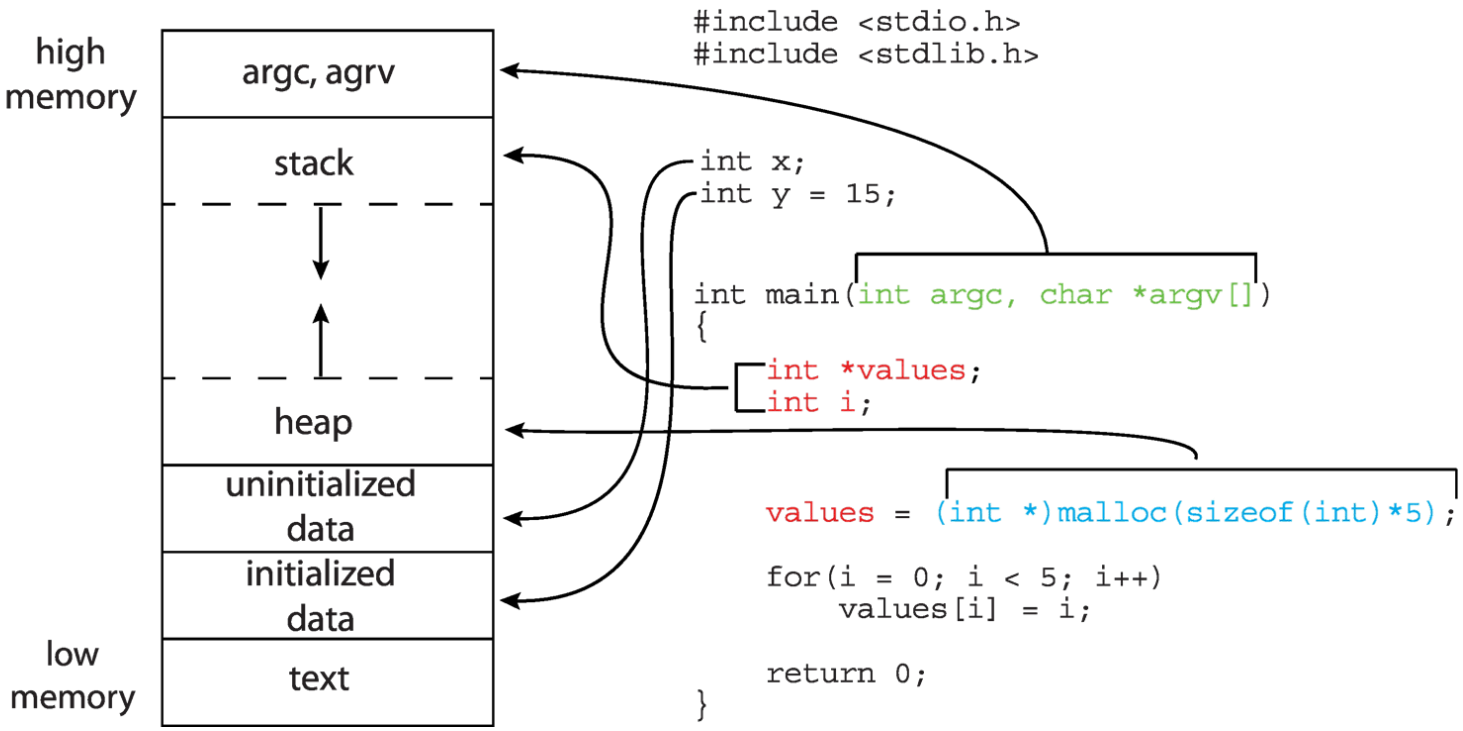




Memory Layout of a C Program

Memory Layout of a C Program:

- ✓ Text : کد کامپایل‌شده(Read-Only)
- ✓ Initialized Data : متغیرهای global/static مقداردهی‌شده
- ✓ BSS : متغیرهای global/static بدون مقدار
- ✓ Heap : حافظه پویا(malloc/new)
- ✓ Stack : پارامترها، local variables، return address





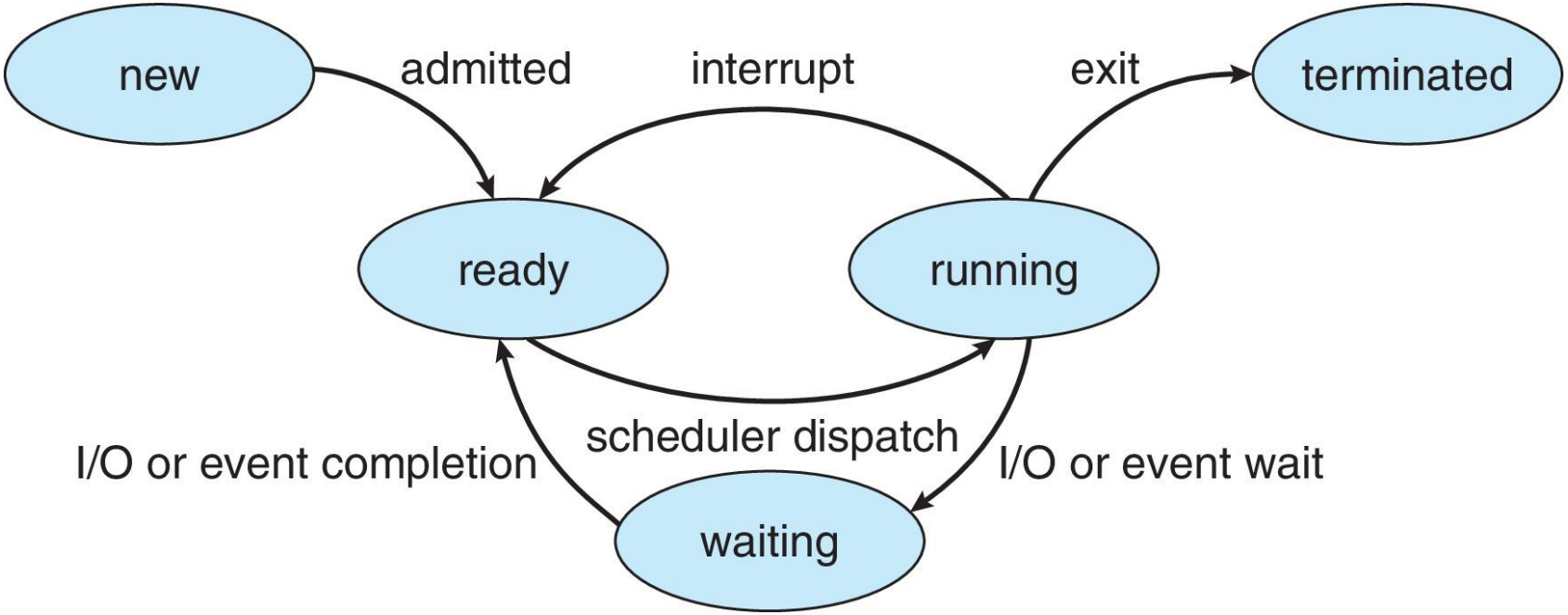
Process State

فرآیندها در زمان اجرا، بین این وضعیت‌ها جابجا می‌شوند:

- New: تازه ایجاد شده
- Ready: آماده برای اجرا
- Running: روی CPU در حال اجرا
- Waiting/Blocked: منتظر I/O
- Terminated: پایان یافته



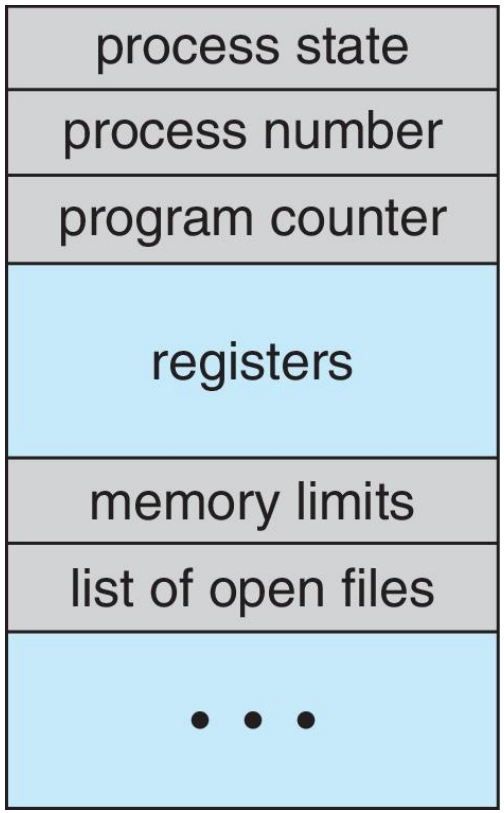
Diagram of Process State





Process Control Block (PCB)

- سیستم عامل اطلاعات هر فرآیند را در ساختاری به نام PCBنگه می‌دارد.
- بلوک کنترل فرایند (PCB) شامل:
  - Process ID
  - Program Counter
  - CPU Registers
  - Scheduling Info
  - Memory Management Info
  - I/O Info
- بدون PCBسیستم‌عامل نمی‌تواند اجرای فرآیندها را مدیریت کند.





Process Scheduling

- زمان‌بند فرآیند (Process Scheduler) وظیفه دارد انتخاب کند کدام فرآیند بعدی روی CPU اجرا شود.
- در یک سیستم چندوظیفه‌ای (Multitasking)، ده‌ها یا صدها فرآیند منتظرند، بنابراین سیستم‌عامل باید یک انتخاب بهینه انجام دهد.

- هدف زمان‌بندی چیست؟
  - حداکثر استفاده از CPU به عبارتی (Maximize CPU Utilization)
  - جابجایی سریع فرآیندها روی CPU
  - جلوگیری از بیکار ماندن پردازنده
  - اجرای همزمان‌گونه‌ی فرآیندها (برای کاربر)



Process Scheduling

- اگر Schedulerبه خوبی کار نکند:
  - CPUمدت زیادی بیکار می‌ماند
  - فرآیندها معطل می‌شوند
  - برنامه‌ها کند اجرا می‌شوند
  - تجربه کاربر ضعیف می‌شود
- Schedulerمغز سیستم‌عامل است برای تصمیم‌گیری "چه فرایندی بعدی اجرا شود".



scheduling queues

- سیستم‌عامل برای مدیریت فرآیندها از صف‌ها (Queues) استفاده می‌کند.

1) Ready Queue(صف آماده به اجرا)

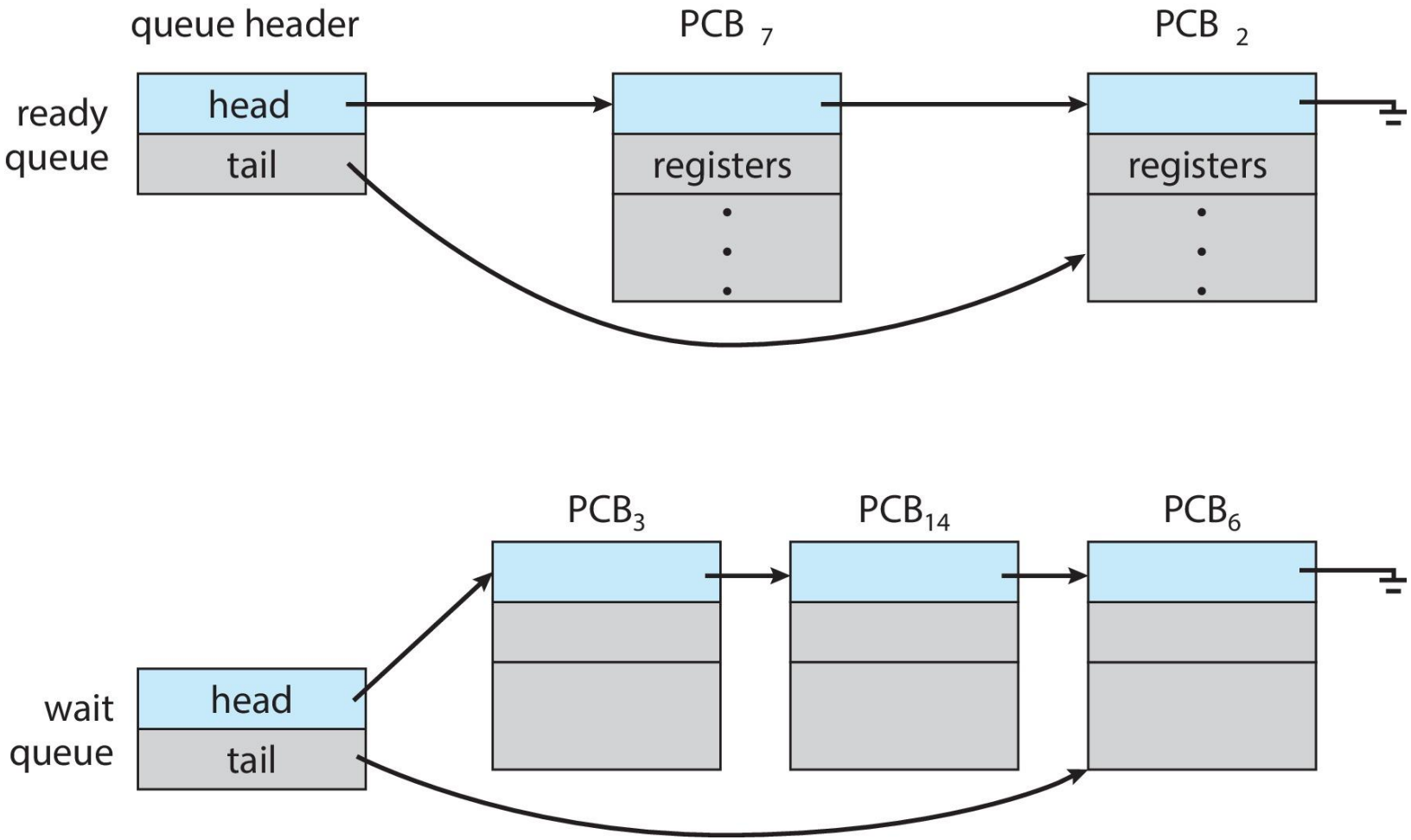
- شامل تمام فرآیندهایی که در حافظه اصلی هستند و آماده‌اند که CPUبه آن‌ها زمان دهد و منتظر نوبتند.

2) Wait Queue یا Device Queue(صف انتظار برای رویداد)

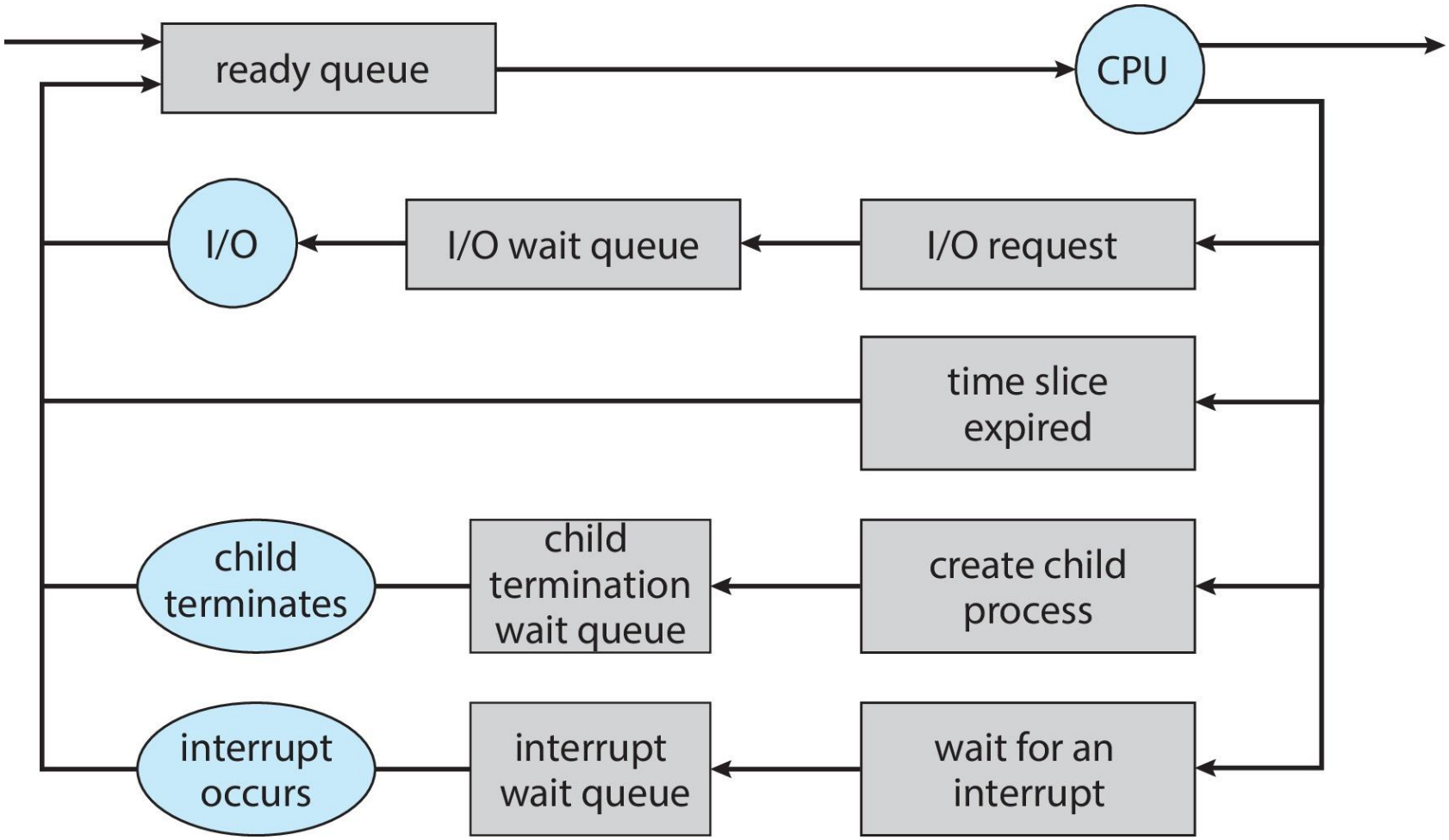
- فرآیندهایی که منتظر یک رویداد هستند:
  - تکمیل ورودی/خروجی (I/O)
  - آزاد شدن یک منبع
  - دریافت یک سیگنال یا پیام
- این فرآیندها نمی‌توانند اجرا شوند تا رویداد موردنظر اتفاق بیفتد.



Ready and Wait Queues



Representation of Process Scheduling





scheduling queues

- فرآیندها بین صف‌ها جابه‌جا می‌شوند:

```
Ready → Running → Waiting → Ready → Running → ...
```

- دلیل استفاده از صف‌ها:
  - جلوگیری از بی‌نظمی در مدیریت فرآیندها
  - افزایش کارایی CPU
  - جلوگیری از گرسنگی (Starvation)
  - امکان اجرای همزمان فرآیندهای متعدد
  - جدا کردن فرآیندهای آماده از فرآیندهای منتظر

- مدیریت صف‌ها باعث می‌شود سیستم‌عامل بتواند ده‌ها فرآیند را بدون دخالت کاربر مدیریت کند.



scheduling queues

- کاربر گوگل کروم را اجرا می‌کند:

1. Chrome در Ready Queue
2. CPU آن را اجرا می‌کند → Running
3. نیاز به بارگذاری صفحه وب دارد → می‌رود Wait Queue (Network I/O)
4. وقتی داده از اینترنت رسید → دوباره وارد Ready Queue
5. CPU دوباره آن را اجرا می‌کند.

- این چرخه در کسری از ثانیه ده‌ها بار رخ می‌دهد.



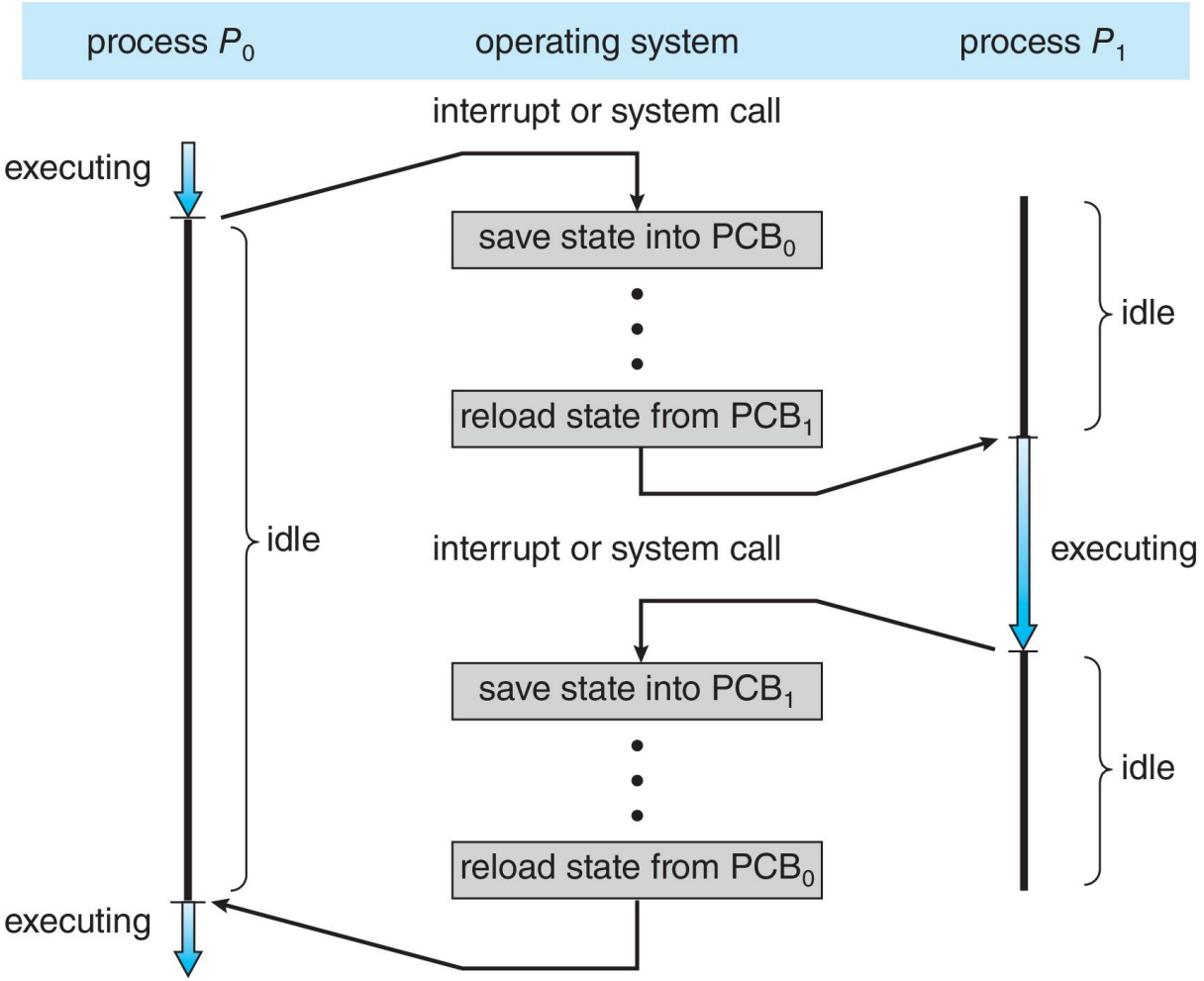
Context Switch

- Context Switch زمانی اتفاق می‌افتد که سیستم‌عامل اجرای یک فرآیند را متوقف کند و اجرای فرآیند دیگری را شروع کند.
- برای این‌کار سیستم‌عامل باید:
  - وضعیت فعلی فرآیند قبلی را ذخیره کند.
  - وضعیت فرآیند جدید را بارگذاری کند.
- وقتی فرآیندی متوقف می‌شود، سیستم‌عامل موارد زیر را ذخیره می‌کند:
  - Program Counter (PC): دستور بعدی که باید اجرا شود
  - CPU Registers: مقادیر لحظه‌ای پردازنده
  - Stack Pointer: وضعیت پشته
  - Memory Management Info: اطلاعات مدیریت حافظه
  - Process State : وضعیت فرآیند(Ready, Waiting, …)
  - PCB : همه اطلاعات ذخیره در این ساختار قرار می‌گیرد
- سیستم‌عامل باید این موارد را دقیقاً ذخیره کند تا بعداً برنامه بتواند بدون مشکل ادامه دهد.



CPU Switch From Process to Process

- Context Switch هر بار که فرآیندی وارد یا خارج از صف‌ها می‌شود، سیستم‌عامل یک انجام می‌دهد تا وضعیت قبلی ذخیره و بعدی بازیابی شود.





Context Switch

- Context switch=زمان تلف‌شده توسط CPU بدون اجرای برنامه واقعی
- زمان زیادی نیست، ولی مهم است... اگر تعداد context switch زیاد شود آنگاه سیستم کند می‌شود.

- Context Switch هزینه بر است اما برای Multitasking ضروری است.
- هرچه سیستم عامل و PCB پیچیده‌تر باشند، زمان تعویض متن طولانی‌تر خواهد بود.
- تعویض متن در زمانهای زیر اتفاق می افتد:
  - زمان عملیات I/O
  - Interrupt
  - پایان سهم زمانی Time Slice
  - توقف یا پایان فرآیند



Operations on Processes

- سیستم باید سازوکارهایی را برای موارد زیر فراهم کند:
  - ایجاد فرآیند
  - خاتمه فرآیند



Process Creation

- در سیستم‌عامل‌ها، یک فرآیند می‌تواند فرآیند دیگری ایجاد کند.
- این‌کار با System Callهایی مثل fork()، CreateProcess()انجام می‌شود.

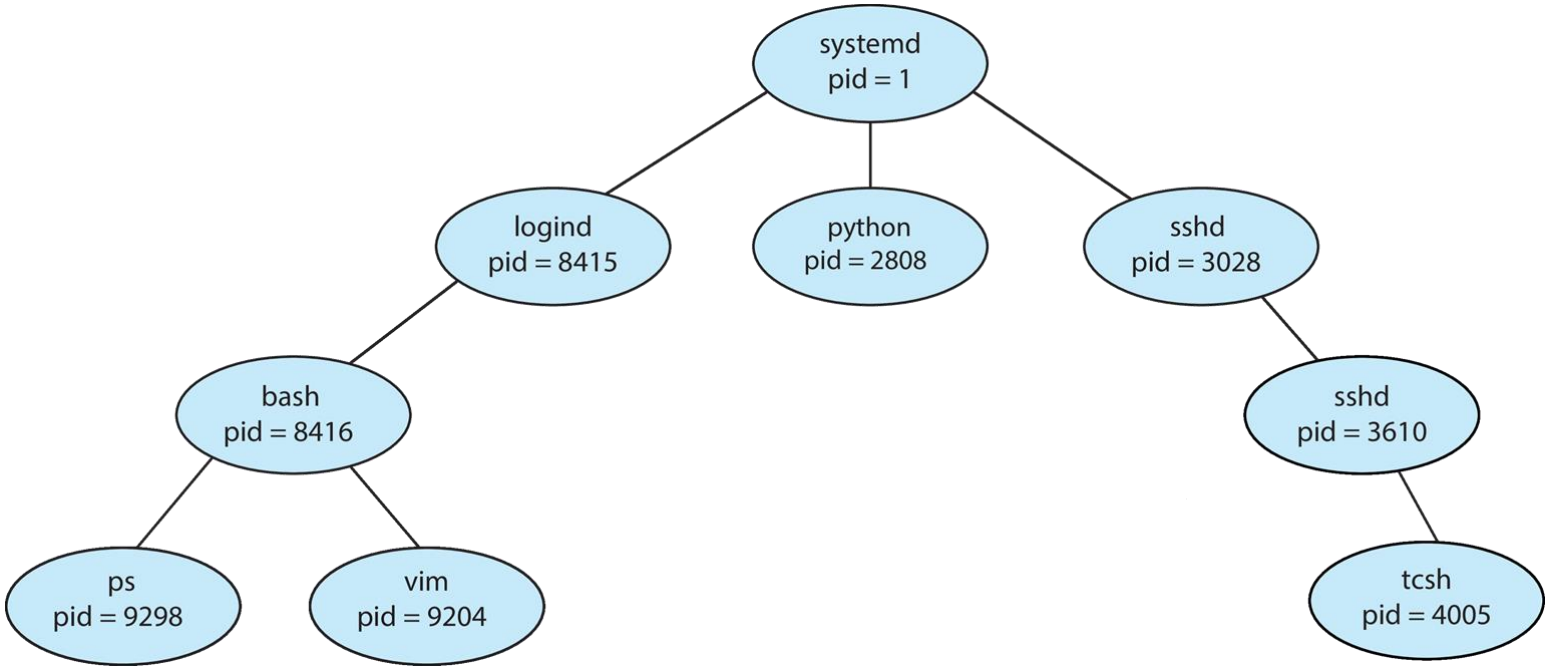
- فرآیند والد (Parent Process): فرآیندی که فرآیند جدیدی را ایجاد می‌کند.
- فرآیند فرزند (Child Process): فرآیند تازه ساخته‌شده.

- هر فرآیند یک PIDدارد (Process ID) که برای شناسایی آن استفاده می‌شود.



A Tree of Processes in Linux

- وقتی والدها فرزندان ایجاد می‌کنند، سیستم یک ساختار درختی تشکیل می‌دهد.
- در ویندوز چنین درختی محدودتر است، ولی وجود دارد.





Resource Sharing Models

- وقتی یک فرآیند، فرزندی ایجاد می‌کند، منابع بین آن‌ها می‌توانند به شکل‌های مختلفی تقسیم شوند.
- سه مدل اشتراک منابع:
  1) هر دو از فایل‌ها و حافظه مشترک استفاده می‌کنند.
  2) فرزند محدودتر است.
  3) فرزند کاملاً مستقل از والد است.
- این تفاوت بین سیستم‌عامل‌ها قابل انتخاب یا قابل پیکربندی است.



Execution Options

دو مدل اجرای فرایندهای والد و فرزند:

1) اجرای همزمان (Concurrent Execution)

- والد و فرزند همزمان اجرا می‌شوند.

2) اجرای ترتیبی (Sequential Execution):

- والد منتظر می‌ماند تا فرزند پایان یابد.

- در یونیکس غالباً هر دو گزینه وجود دارد:
  - `wait()`برای توقف والد/ بدون آن اجرای همزمان



Process Termination

- فرآیند ممکن است با دو روش پایان یابد:

1) پایان طبیعی (Normal Exit)

- برای مثال: کامل شدن اجرای برنامه
- فراخوانی `exit()`
- برگشت از تابع `main`

2) پایان غیرعادی (Abnormal Termination)

- خطا مانند خطای محاسباتی ،خطای ورودی / خروجی،خطای حفاظت
- پایان دادن توسط والد
- پایان دادن توسط سیستم‌عامل



Parent-initiated Termination

- والد ممکن است اجرای فرزند را با فراخوانی سیستمی `abort()` پایان دهد؛زیرا:
  - فرزند بیش از حد منابع مصرف می‌کند
  - برنامه فرزند وارد حلقه بی‌نهایت می‌شود
  - فرزند کاری انجام می‌دهد که والد مجاز نمی‌داند
  - والد در حال خاتمه است → فرزندها باید حذف شوند

- عملیاتی که انجام می‌شود:
  - سیستم‌عامل تمام منابع فرزند را آزاد می‌کند
  - فرزند به وضعیت `Zombie` می‌رود تا والد وضعیت آن را دریافت کند
  - سپس از جدول فرآیند حذف می‌شود



Cascading Termination

- بسیاری از سیستم‌عامل‌ها (مثل UNIX/Linux) حالتی دارند که اگر والد بمیرد، تمام فرزندانش نیز پایان می‌یابند. این کار برای جلوگیری از:
  - orphan processes(فرآیندهای یتیم/ بی‌والد)
  - مصرف بی‌دلیل منابع
  - اجرای برنامه‌های ناخواسته
- رفتار UNIX : اگر والد بمیرد:
  - فرزندان ابتدا از آن جدا می‌شوند (become orphan)
  - سپس به فرآیند init/systemd سپرده می‌شوند
  - و اگر لازم باشد پایان داده می‌شوند
- رفتار Windows:اگر والد بمیرد:
  - در ویندوز ارتباط والد–فرزند ضعیف‌تر است.
  - پایان والد معمولاً باعث پایان خودکار فرزند نمی‌شود مگر برنامه‌نویس Explicitتعریف کند.



Process Termination and wait()

- تابع `wait()` در والد استفاده می‌شود تا منتظر پایان فرزند بماند.

- وقتی فرزند تمام می‌شود:
  - وضعیت خروج (`exit status`) به والد ارسال می‌شود
  - فرزند به `Zombie` تبدیل می‌شود
  - وقتی والد `wait()` را دریافت می‌کند → `Zombie` پاک می‌شود



Orphans & Zombies

Orphan Process

- فرآیندی که والدش قبل از او پایان یافته است.
- در لینوکس → این فرآیند تحت مراقبت `init/systemd` قرار می‌گیرد.

Zombie Process

- فرآیندی که پایان یافته ولی اطلاعاتش در جدول فرآیند پاک نشده، چون والد هنوز `wait()` نکرده است.
- اگر والد `wait()` را انجام ندهد → جدول فرآیند از `Zombie`ها پر شده و سیستم کند می‌شود



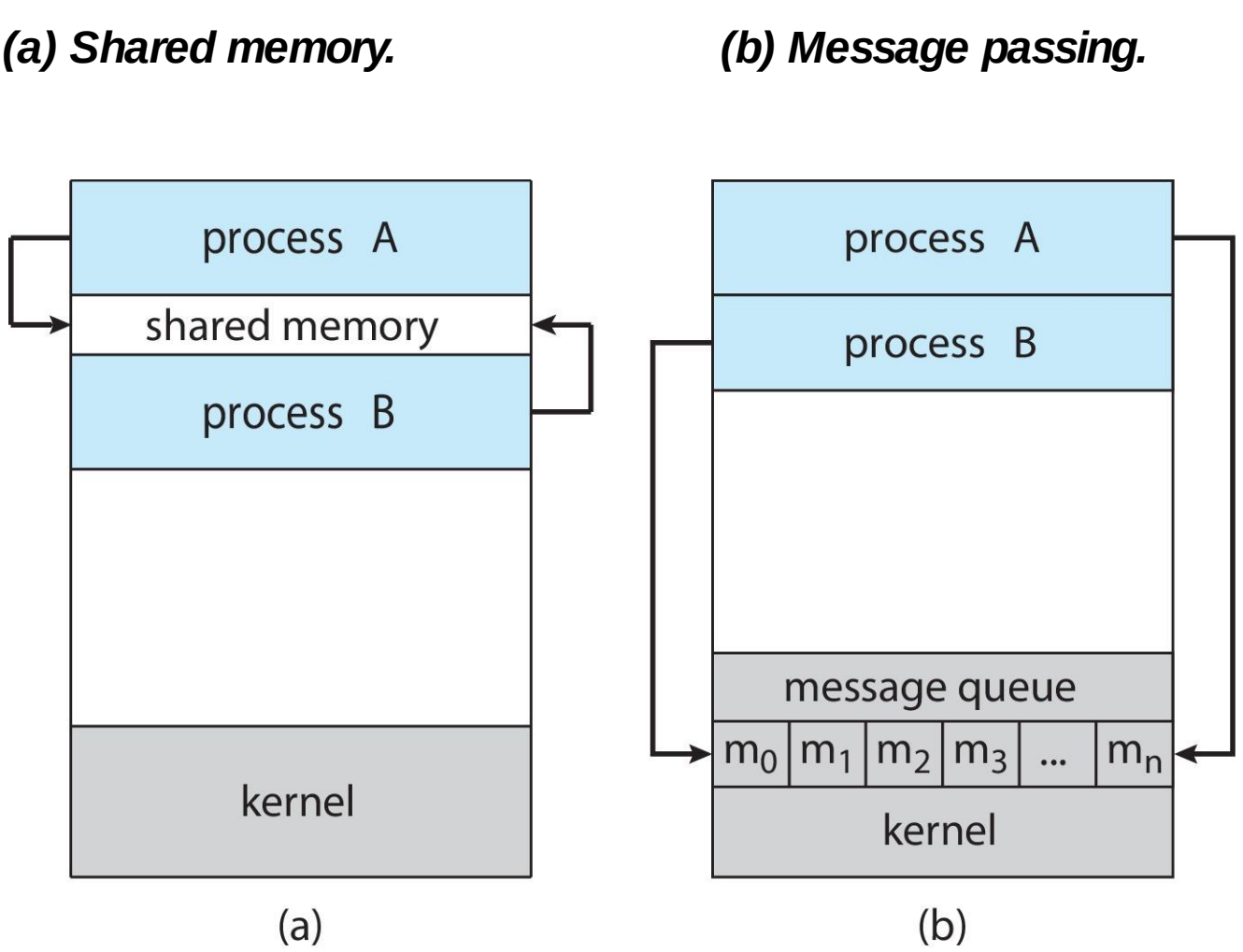
Interprocess Communication (IPC)

- فرآیندها نیاز دارند با هم ارتباط برقرار کنند؛برای:
  - اشتراک داده
  - هماهنگ‌کردن عملیات
  - ارسال درخواست
  - همکاری در یک کار مشترک

- IPCدر دو مدل است:
  1. Shared Memory(حافظه مشترک)
  2. Message Passing(ارسال پیام)



Communications Models





Shared Memory Model

- بخش‌هایی از حافظه که بین دو یا چند فرآیند مشترک استفاده می‌شود.

- ویژگی‌ها:
  - سرعت بالا
  - نیاز به همگام‌سازی (synchronization)
  - معمولاً با semaphore یا mutex کنترل می‌شود

- مثال کاربردی:
  - مرورگر (Chrome)
  - بانک‌های اطلاعاتی
  - سیستم‌های مدیریت Cache



Message Passing Model

- در این مدل فرآیندها بجای اشتراک حافظه، پیام رد و بدل می‌کنند.
- مناسب برای:
  - سیستم‌های توزیع‌شده
  - پردازه‌های روی ماشین‌های مختلف
  - معماری‌های میکروکرنل (مثل Mach, Minix)
- دو نوع پیام:
  - `send(message)`
  - `receive(message)`



Message Passing: Direct & Indirect Communication

1) ارتباط مستقیم (Direct Communication)

- فرستنده و گیرنده یکدیگر را با نام صدا می‌زنند:

```
send(P2, message)
receive(P1, message)
```

- عیب : coupling زیاد (وابستگی شدید)

2) ارتباط غیرمستقیم (Indirect Communication)

- پیام‌ها در mailboxes / ports ارسال می‌شوند:

```
send(mailbox1, message)
receive(mailbox1, message)
```

- مزیت : استقلال و انعطاف بالا



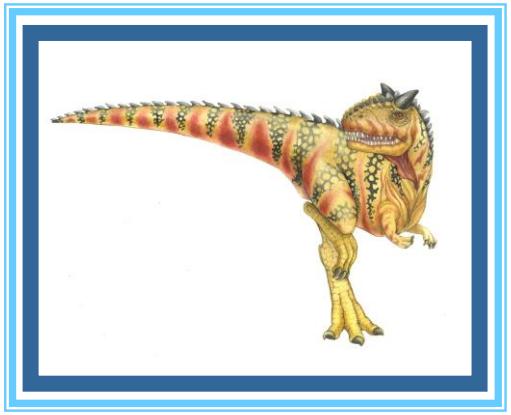
Synchronization in Message Passing

- ارسال پیام ممکن است مسدود کننده یا غیرمسدود کننده باشد.
- مسدود شدن به عنوان همزمان در نظر گرفته می شود.
  - **ارسال مسدود کننده** :فرستنده تا زمانی که پیام دریافت نشود مسدود می شود.تا ارسال دیگری انجام نشود.
  - **دریافت مسدود کننده:** گیرنده تا زمانی که پیامی در دسترس نباشد مسدود میشود.
- غیر مسدود شدن به عنوان ناهمزمان در نظر گرفته می شود.
  - **ارسال غیر مسدود کننده:** فرستنده پیام را ارسال می کند و ادامه می دهد.
  - **دریافت غیر مسدود کننده**: گیرنده دریافت می کند:یک پیام معتبر، یا پیام تهی(null)
  - این مدل در سیستم‌های real-time کاربرد زیادی دارد چون پردازه معطل نمی‌ماند.
- ترکیب های مختلفی امکان پذیر است:
  - اگر هم ارسال و هم دریافت مسدود کننده باشند، یک دیدار (rendezvous)داریم.



البته این مدل برای سیستم‌هایی که latency مهم است؛ نامناسب می‌باشد.

End of Chapter 3



Silberschatz, Galvin and Gagne ©2013